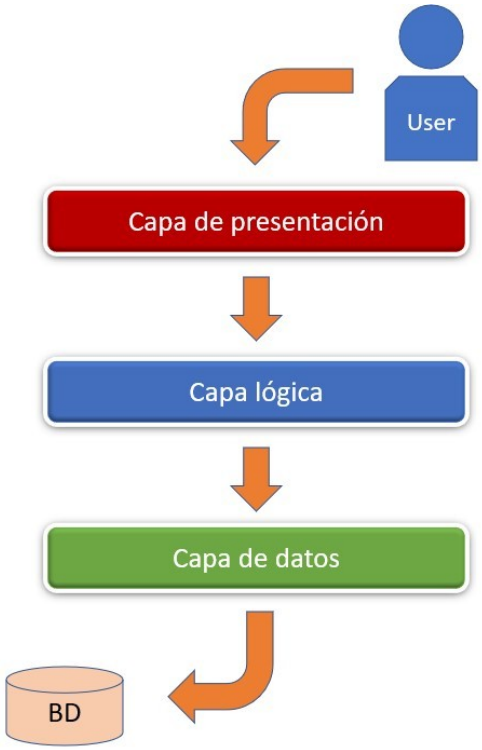
Tema 7.
Servicios Web

Introducción

- Las aplicaciones web se modelan a través de lo que se conoce como modelo de capas –>
  - La capa de presentación se encarga de la interfaz de usuario de la aplicación, lo que el usuario ve y cómo interactúa.
  - La capa de lógica o de negocio se encarga de la lógica de la aplicación. Esta capa es responsable de realizar cualquier cálculo, procesamiento y toma de decisiones necesarias para hacer que la aplicación funcione correctamente.
  - La capa de datos se encarga de la gestión de los datos de la aplicación. Esta capa se encarga de almacenar, buscar, actualizar y eliminar lo datos de la aplicación, así como proporcionar los datos a la capa de lógica para su procesamiento.
    - ✓ Capa de Servicio
    - ✓ Capa de Modelo



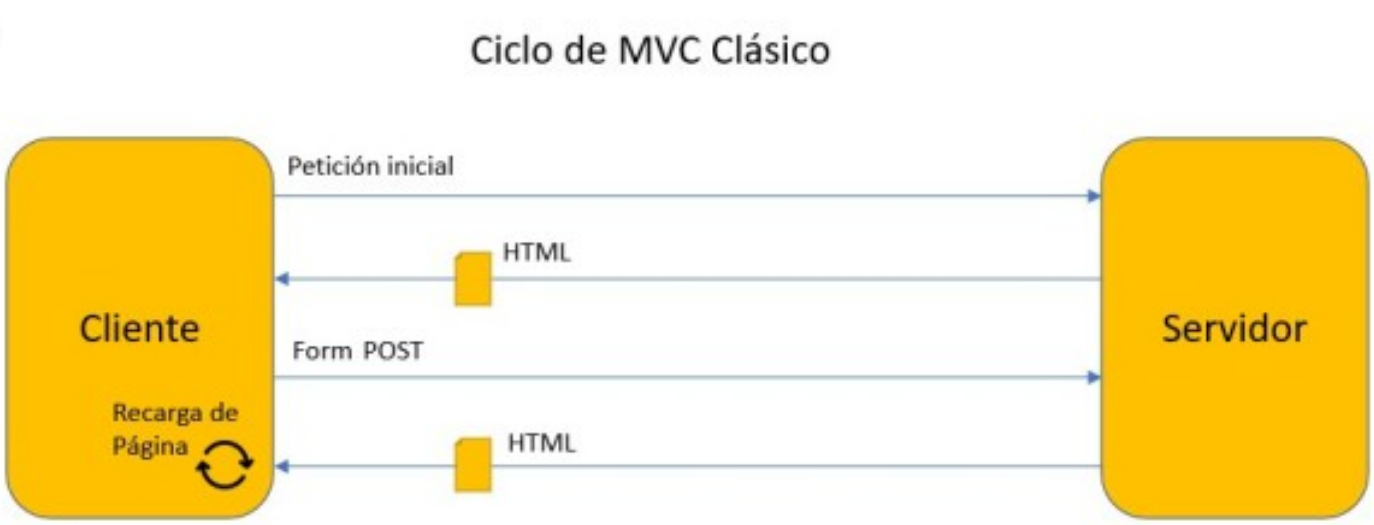
Introducción

- Para implementar este modelo de capas podemos optar por dos soluciones, como pueden ser el **MVC o Modelo Vista Controlador clásico** y una variante de este modelo donde la vista se implementaría a través del modelo **SPA (Single Page Application)**.
- Estas dos soluciones son las que más se están utilizando actualmente para el desarrollo web.

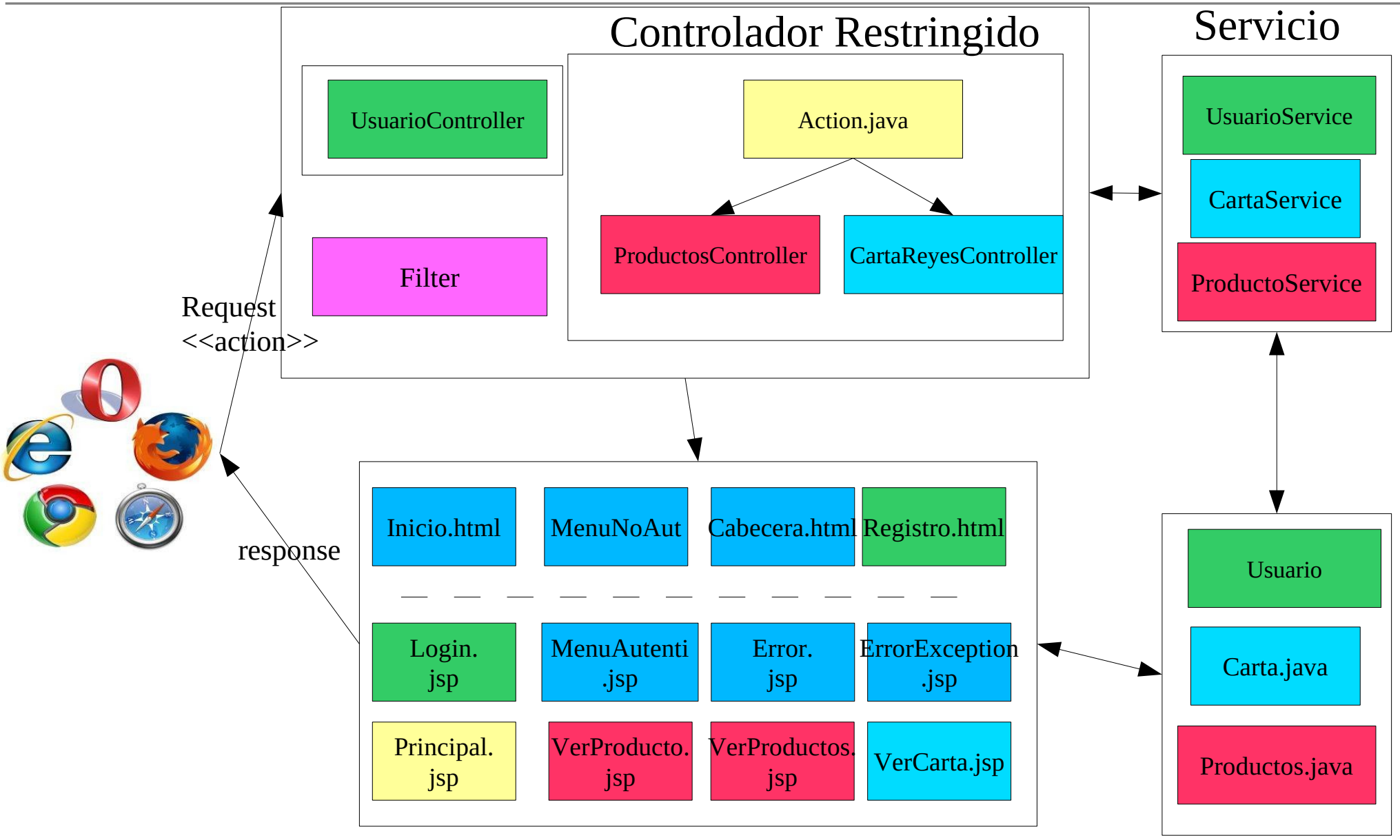


Introducción

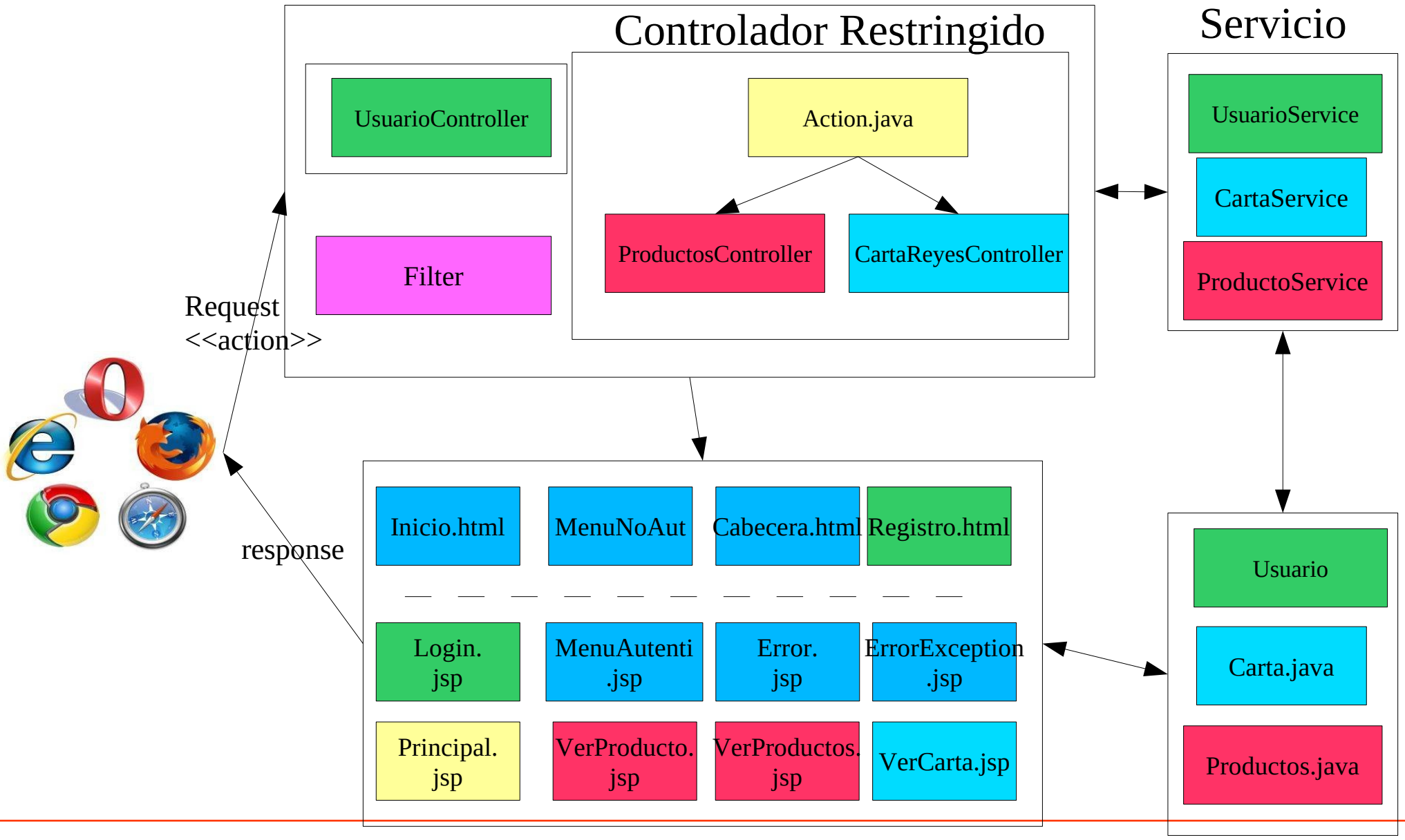
- El Modelo Vista Controlador (MVC) clásico es un patrón de diseño de aplicaciones de software que se basa en una separación de responsabilidades entre tres elementos: el modelo, la vista y el controlador.
- Cuando un usuario hace una solicitud, el controlador se encarga de recibir y procesar la solicitud, luego obtiene los datos necesarios del modelo y luego los envía a la vista para que los muestre.



Introducción



Introducción

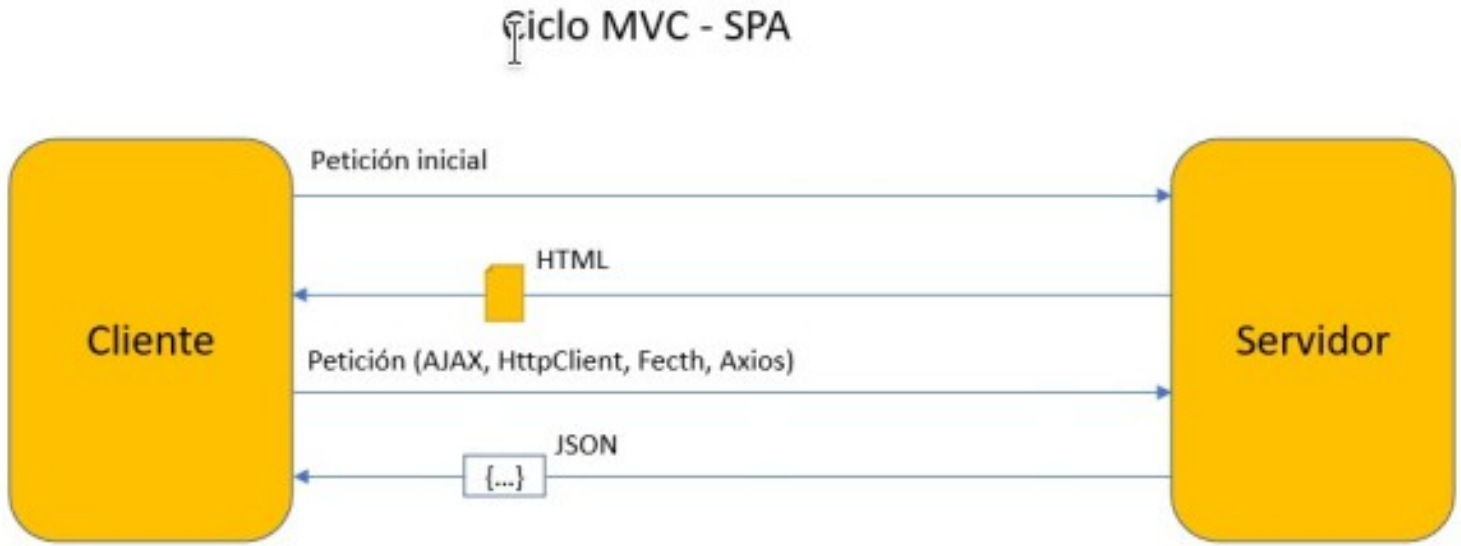


Introducción

- El Modelo Vista Controlador donde la vista es una Single Page Application es un patrón de diseño de aplicaciones web similar al MVC clásico, excepto que la vista es una aplicación de página única.
- Esto significa que toda la lógica de presentación de la aplicación se almacena en una sola página web.
- Esta lógica se encarga de manejar el flujo de la aplicación, la navegación entre páginas y la gestión de estados.
- El controlador sigue siendo responsable de recibir las solicitudes del usuario y procesarlas, pero en lugar de mostrar los datos en la vista, los envía de nuevo a la aplicación para que los procese.
- La SPA a su vez se encarga de obtener los datos necesarios del modelo y mostrarlos en la vista.

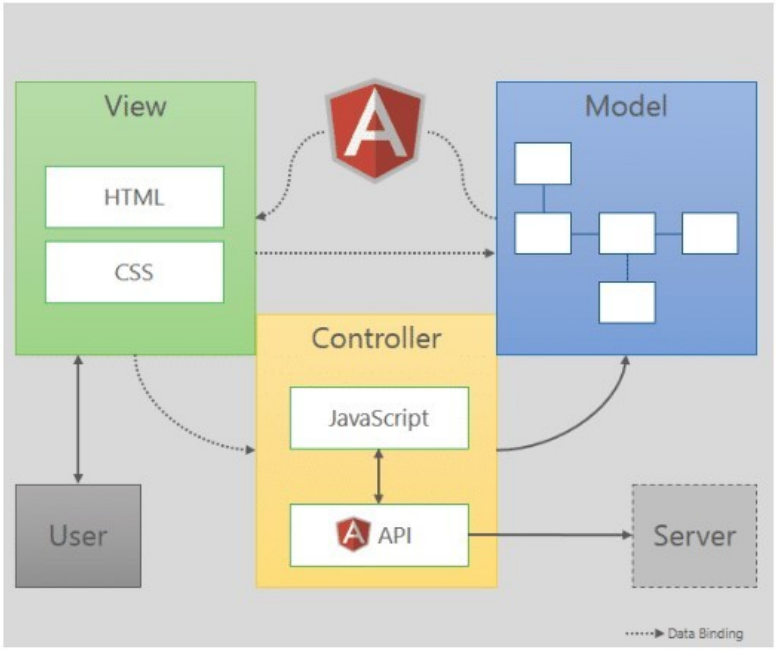
Introducción

- La carga de la página en el cliente se produce una única vez al iniciar la petición y a partir de ese instante solo se realizan peticiones ya sean AJAX o la implementación de peticiones de algún framework específico como Angular (HttpClient) o React (Fetch, Axios), y las respuestas son en formato JSON.



Introducción

- La carga de la página en el cliente se produce una única vez al iniciar la petición y a partir de ese instante solo se realizan peticiones ya sean AJAX o la implementación de peticiones de algún framework específico como Angular (HttpClient) o React (Fetch, Axios), y las respuestas son en formato JSON.
- Estructura de Angular



Introducción

- En esta parte de la asignatura, nos centraremos en el desarrollo de la arquitectura que se encarga de retornar la información solicitada en distintas formatos (txt, html, xml o json) → Servicios Web.



- Queda fuera del ámbito de esta asignatura el desarrollo de toda la parte cliente que se encarga de realizar las peticiones y actualizar el navegador.

Imagen obtenida de: https://programarfacil.com/wp-content/uploads/2015/12/web-services.png

Resumen

- Qué es un servicio web?
- Qué es SOA?
- Arquitectura de un servicio web SOAP
  - SOAP
  - WSDL
  - UDDI
- Arquitectura de un servicio web Rest

Qué es un servicio web?

- W3C: Servicios Web:
  - Son un **conjunto de aplicaciones o de tecnologías con capacidad para interoperar** en la Web. Estas aplicaciones o tecnologías **intercambian datos entre sí con el objetivo de ofrecer unos servicios**. Los proveedores ofrecen sus servicios como procedimientos remotos y los usuarios solicitan un servicio llamando a estos procedimientos a través de la Web.
- Wikipedia: Servicios Web
  - Un servicio Web es una **colección de protocolos y estándares que sirve para intercambiar datos entre aplicaciones**. Distintas aplicaciones de software desarrolladas en lenguajes de programación diferentes y ejecutadas sobre cualquier plataforma pueden utilizar los servicios Web para intercambiar datos en redes de ordenadores como Internet.

Qué es un servicio web?

- De forma coloquial
  - Servicios Web son los que hacen posible que alguien o algo que necesita un servicio y otra que lo ofrezca, puedan llevar a cabo la operación a través de Internet.

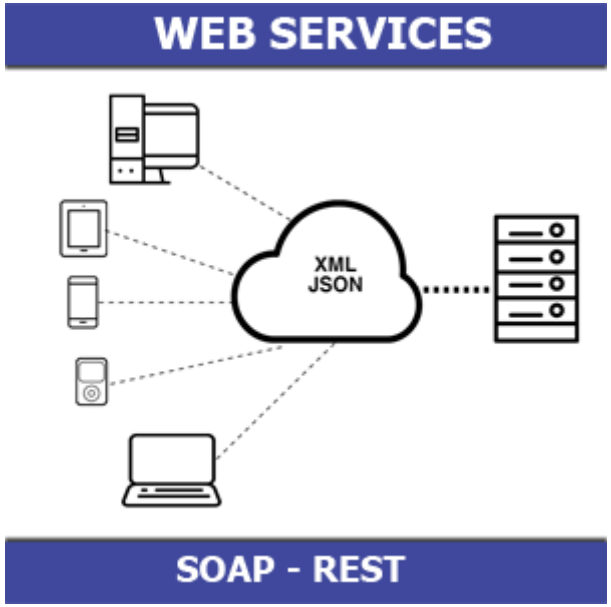


Imagen obtenida de: https://programarfacil.com/wp-content/uploads/2015/12/web-services.png

Qué es un servicio web?

- Objetivos de los servicios web
  - Independencia del lenguaje y de la plataforma
    - ✓ Separación de especificación de la implementación
  - Interoperabilidad
    - ✓ Utilización de estándares: XML, SOAP, WSDL, UDDI…
    - ✓ Utilización de estándares: HTTP, GET, POST,...
  - Acoplamiento débil: Sistemas basados en mensajes
    - ✓ Interacciones síncronas y asíncronas
  - A través de Internet
    - ✓ Sin control centralizado
    - ✓ Utilización de Protocolos establecidos
    - ✓ Consideraciones de seguridad
  - Modularidad y Reusabilidad de servicios
    - ✓ Reutilización dentro de la misma aplicación u en otras

Qué es un servicio web?

- Ventajas
  - ✓ Aportan interoperabilidad entre aplicaciones de software independientemente de sus propiedades o de las plataformas sobre las que se instalen.
  - ✓ Los servicios Web fomentan los estándares y protocolos basados en texto, que hacen más fácil acceder a su contenido y entender su funcionamiento.
  - ✓ Permiten que servicios y software de diferentes compañías ubicadas en diferentes lugares geográficos puedan ser combinados fácilmente para proveer servicios integrados.
- Inconvenientes
  - ✓ Al apoyarse en HTTP, pueden esquivar medidas de seguridad basadas en firewall cuyas reglas tratan de bloquear o auditar la comunicación entre programas a ambos lados de la barrera.
  - ✓ Su rendimiento es bajo si se compara con otros modelos de computación distribuida, tales como RMI (Remote Method Invocation), CORBA o DCOM.

REST

- REpresentational State Transfer

Introducción

- ¿Qué pensáis de WSDL y SOAP? Demasiado complejo??

- ¿Hay algo más “fácil”?
  - También eficiente y escalable

- REST
  - Es la forma en la que trabaja la propia WWW, con un poco de formalización y con cosas que se deben y no se deben hacer
  - Se trata de aplicar los principios que ya sabemos que funcionan, la WWW, para diseñar servicios web

¿Qué es REST?

- Roy Fielding, en su tesis doctoral, trató de responder a las preguntas:
  - ¿Por qué es la WWW tan prevalente y ubicua?
  - ¿Qué hace que la WWW escale?
  - ¿Cómo puedo aplicar la arquitectura de la WWW a mis propias aplicaciones?

- La respuesta fue un conjunto de principios arquitecturales a los que denominó **REpresentational State Transfer** o **REST**

- Es, por tanto, un estilo de construir aplicaciones

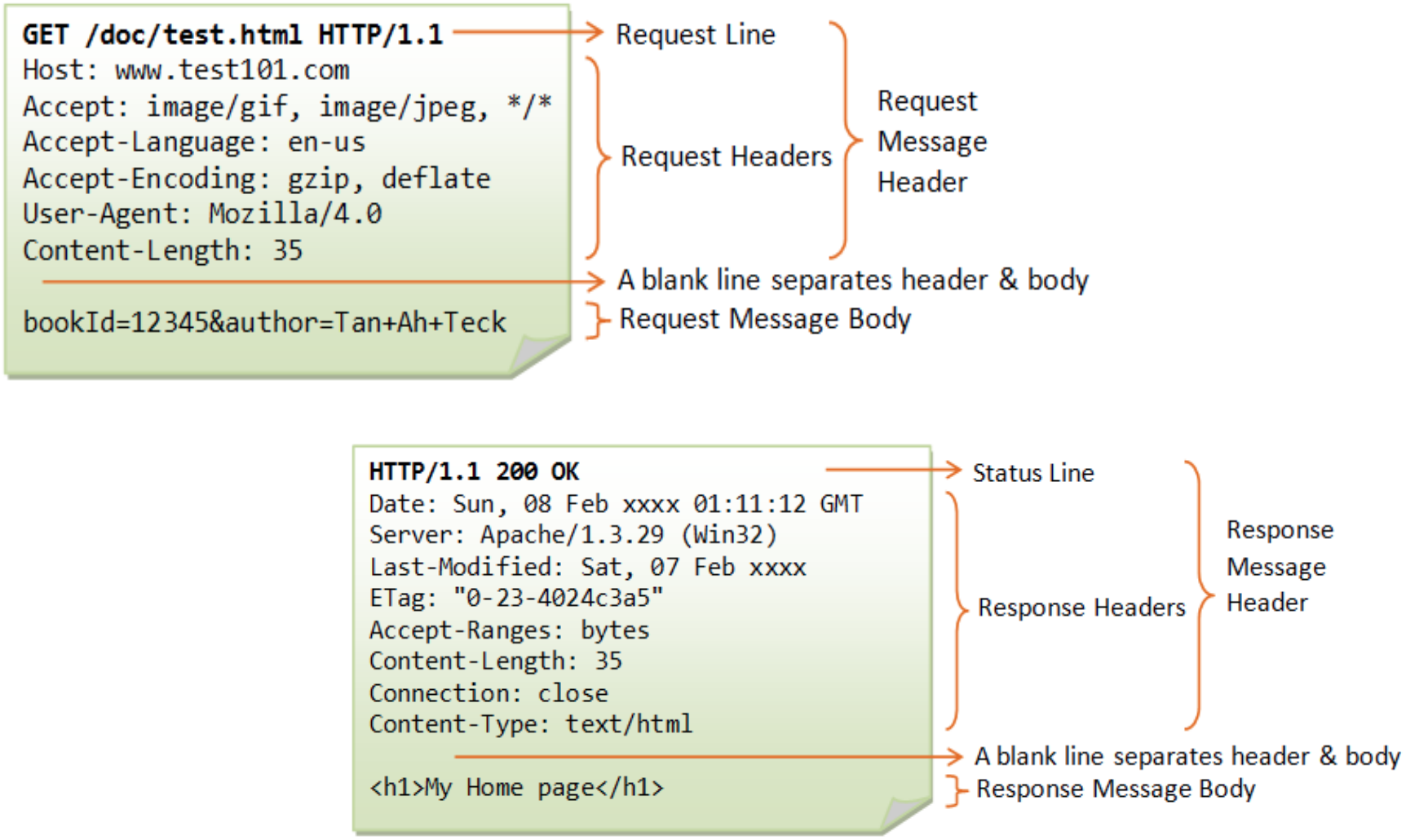
Principios de REST

- Recursos direccionables
  - La abstacción de información clave en REST son los **recursos**, y cada recurso ha de ser accesible a través de una URI (Uniform Resource Identifier)
- Un interfaz uniforme y restringido
  - Uso de un conjunto pequeño y bien definido de métodos para gestionar recursos
- Orientado a la representación
  - La interacción con los recursos es a través de representaciones de dicho servicio
- Comunicaciones sin estado (mensajes autocontenidos)
  - Facilita la escalabilidad
- Hypermedia como motor del estado de la aplicación
  - Los formatos de datos guían la transición entre estados de la aplicación

REST y el renacimiento de HTTP

- REST no está definido sobre un protocolo específico, pero cuando se habla de REST se suele hacer como REST sobre HTTP
- En SOAP y WSDL, HTTP es sólo el protocolo de transporte...
- … pero HTTP tiene muchísimas más características
- HTTP es muy simple y se basa en peticiones de un cliente que están compuestas por:
  - Un método HTTP
  - La localización del recurso que se quiere invocar
  - Un conjunto variable de cabeceras
  - Un cuerpo de mensaje opcional que puede incluir cualquier cosa: HTML, texto plano, XML, JSON, datos binarios

Ejemplo de petición HTTP



Direccionamiento en Rest

- Todo objeto o recurso del sistema se puede direccionar a través de un identificador único

- En REST, la direccionabilidad se gestiona a través de URIs

```
scheme://host:port/path?queryString#fragment
```

- Por ejemplo:

```
http://example.com/customers?lastName=Burke&zipcode=02115
```

JAX-RS

- La creación de servicios web REST se realizó durante muchos años con servlets

- En 2008 se definió una nueva especificación para simplificar la implementación de este tipo de servicios → JAX-RS

- JAX-RS framework
  - Usa anotaciones
    - ✓ Asocian patrones en las URI y operaciones HTTP con métodos de una clase Java
  - Proporciona herramientas para procesar diferentes tipos de mensajes
  - Excepciones específicas
  - Ayudas para la negociación del protocolo HTTP

Jersey

- Jersey es la implementación de referencia para la especificación Jax-RS(JSR 311).
- La implementación Jersey ofrece una biblioteca para implementar servicios web en un contenedor de servlets Java (Tomcat, Jetty, Jboss,….)
- En el lado del servidor Jersey proporciona una implementación para identificar los recursos REST.
  - En el fichero de configuración web.xml debe registrarse que hay una extensión al contenedor y donde se encuentran los recursos.
- La implementación Jersey también ofrece una biblioteca cliente para comunicarse con un servicio web RESTful.

Jersey

- Este servlet analiza la solicitud HTTP entrante y selecciona la clase y el método correcto para responder a esta solicitud.
- Esta selección **se basa en las anotaciones de la clase** y los métodos.
- Una aplicación web REST consiste, por tanto, en un conjunto de clases Java y una clase con los servicios que proporciona.
- Estos dos tipos se mantienen generalmente en diferentes paquetes.
- JAX-RS apoya la creación de XML y JSON a través de la JAXB (JAXB).

Jersey

- Las anotaciones más importantes de JAX-RS (existen más):

Anotación	Descripción
PATH	Establece la ruta a la base URL + / your_path. La URL base se basa en su nombre de la aplicación, el servlet y el patrón de URL del `web.xml` archivo de configuración.
POST	Indica que el siguiente método responderá a una solicitud HTTP POST.
GET	Indica que el siguiente método responderá a una solicitud HTTP GET.
PUT	Indica que el siguiente método responderá a una solicitud HTTP PUT.
DELETE	Indica que el siguiente método responderá a una solicitud DELETE HTTP.
Produces (MediaType.TEXT_PLAIN [, más-types])	Define qué tipo MIME usará un método anotado con GET. "text / plain", "application / xml" o "application / json".
Consumes (Tipo [, más tipos])	Define qué tipo MIME será usada en la petición del servicio
PathParam	Se utiliza para establecer parámetros en la llamada

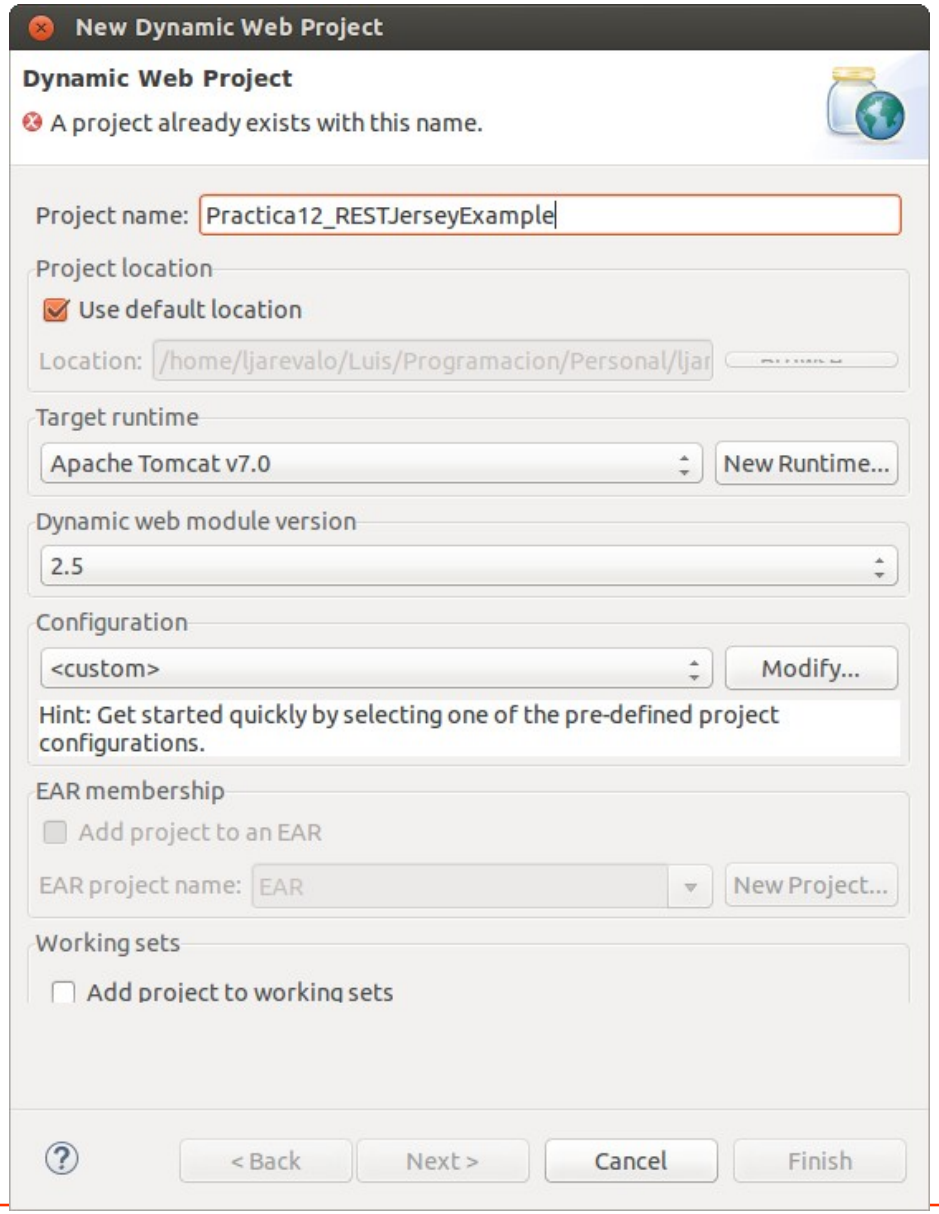
Jersey (Instalación)

- Descargar Jersey de la web: http://jersey.java.net
  - Versión actual: 3.1.1. que habitualmente se utiliza mediante maven, aunque se puede descargar)
    - ✓ https://repo1.maven.org/maven2/org/glassfish/jersey/bundles/jaxrs-ri/3.1.1/jaxrs-ri-3.1.1.zip
- Eclipse y Tomcat
  - Usaremos Tomcat como contenedor de servlets
  - Hay que preparar Eclipse para desarrollo web
    - ✓ Java EE Developer Tools
    - ✓ Java Web Developer Tools
    - ✓ JAX-WS*
    - ✓ JST server *

Creación de un servicio web REST

- Crear un “Dynamic web project”
  - Apache Tomcat: 10
  - Dynamic web module: 5.0

- Se proporciona un .zip con las librerías necesarias.



Creación de un servicio web REST

- **Copiar los jar de Jersey en WEB-INF/lib y esta clase java**

```
@Path("/HolaMundo")
public class Hello {
  // Texto
  @GET
  @Produces(MediaType.TEXT_PLAIN)
  public String sayPlainTextHello() {return "Hello Jersey (TEXTO)";}
  // XML
  @GET
  @Produces(MediaType.TEXT_XML)
  public String sayXMLHello() {return "<?xml version=\"1.0\"?>" + "<hello> Hello
Jersey (XML)" +"</hello>";}
  // HTML
  @GET
  @Produces(MediaType.TEXT_HTML)
  public String sayHtmlHello() {
   return "<html> " + "<title>" + "Hello Jersey" + "</title>"+
   "<body><h1>" + "Hello Jersey (HTML)" + "</body></h1>" + "</html> ";
  }
  // JSON
  @GET
  @Produces(MediaType.APPLICATION_JSON)
  public String sayJSONHello() {return "HelloWorld";}
}
```

Operación

Formato de salida

Operación

Formato de salida

Operación

Formato de salida

Importante: No puede haber dos procedimientos que respondan a la misma operación (Get) y produzcan el mismo resultado (mismo path)
Si tuvieramos dos métodos con GET y TEXT_PLAIN → ERROR

Creación de un servicio web REST

- Se debe modificar el fichero web.xml para que responda a las llamadas REST y conozca quién debe responder a ellas
  - Si no esta, se genera con botón derecho → JavaEE Tools

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app ….>
   <display-name>Ejemplo_RESTJerseyExample</display-name>
   <servlet>
         <servlet-name>Jersey REST Service</servlet-name>
         <servlet-class>org.glassfish.jersey.servlet.ServletContainer</servlet-class>
         <init-param>
                  <param-name>jersey.config.server.provider.packages</param-name>
                  <param-value>es.unex.cum.tw.rest.server</param-value>
         </init-param>
         <load-on-startup>1</load-on-startup>
    </servlet>
    <servlet-mapping>
         <servlet-name>Jersey REST Service</servlet-name>
         <url-pattern>/rest/*</url-pattern>
    </servlet-mapping></web-app>
```

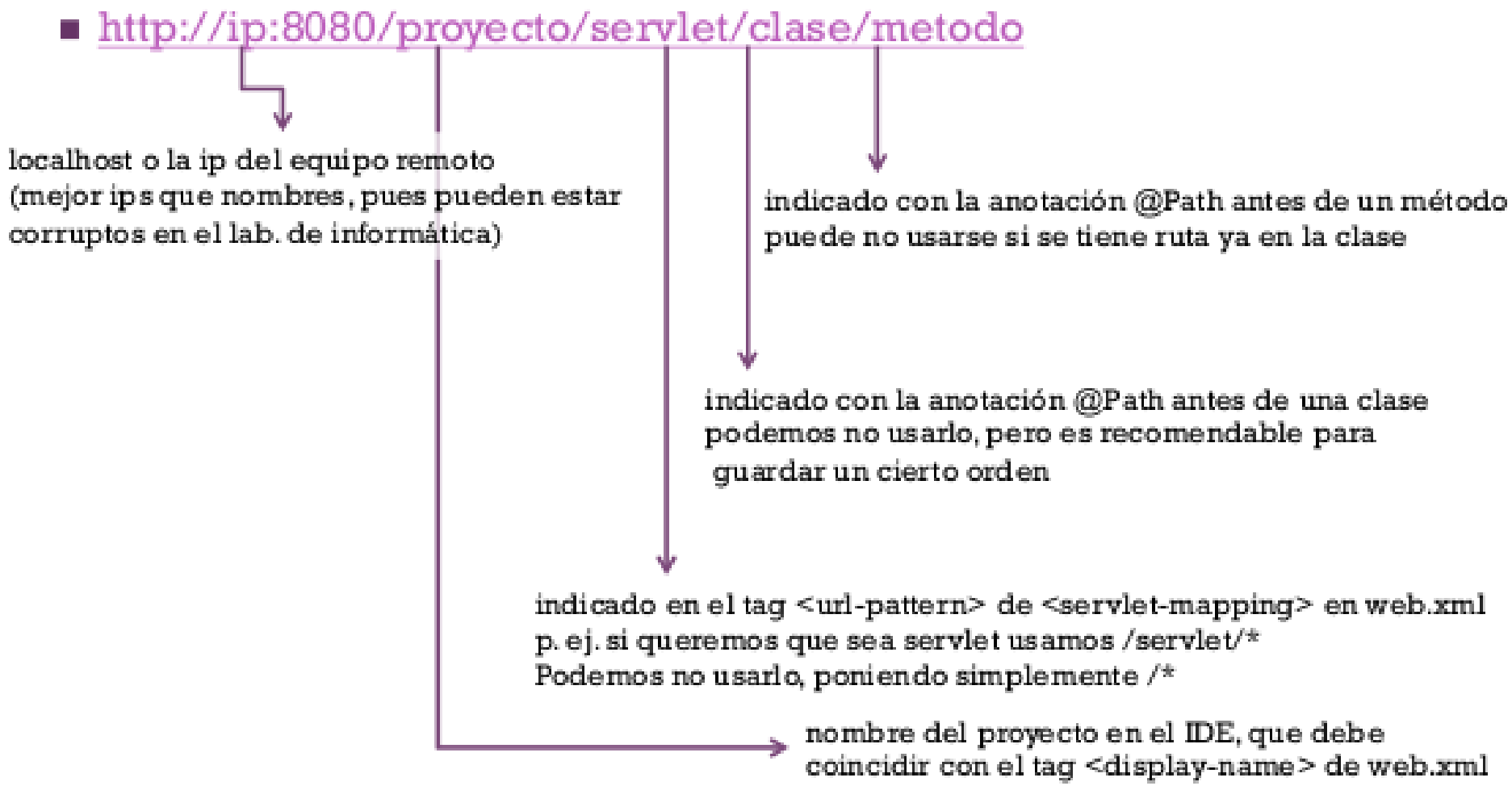
Configuración para que sea Jersey

Classes donde están los servicios

URLs donde están los servicios

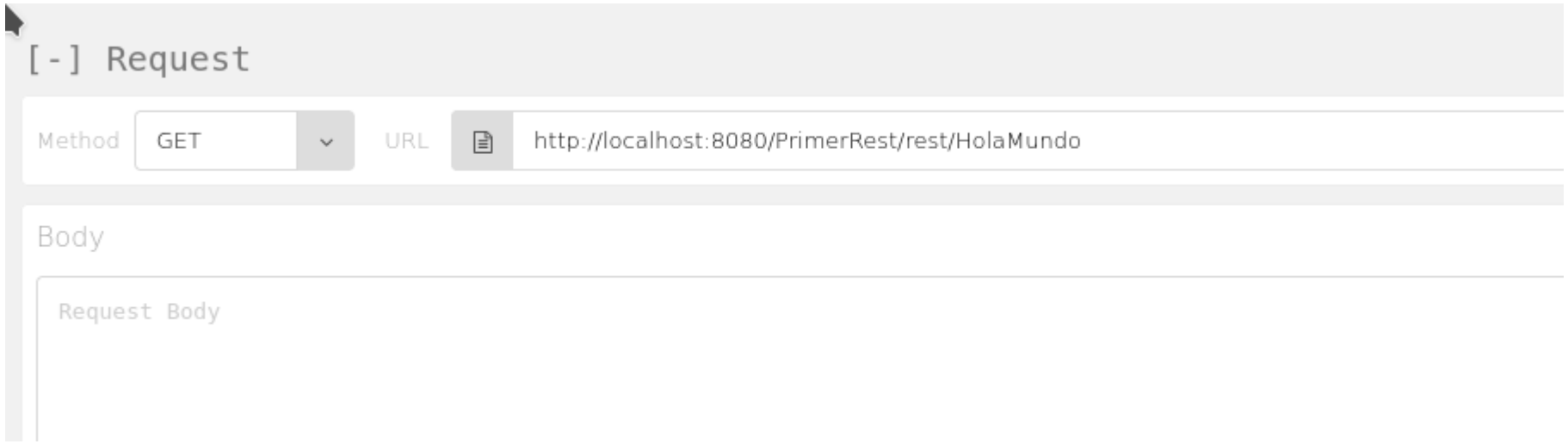
Creación de un servicio web REST

- La URL base de este servlet es:
  - http://localhost:8080/Ejemplo_RESTJerseyExample/rest/hello



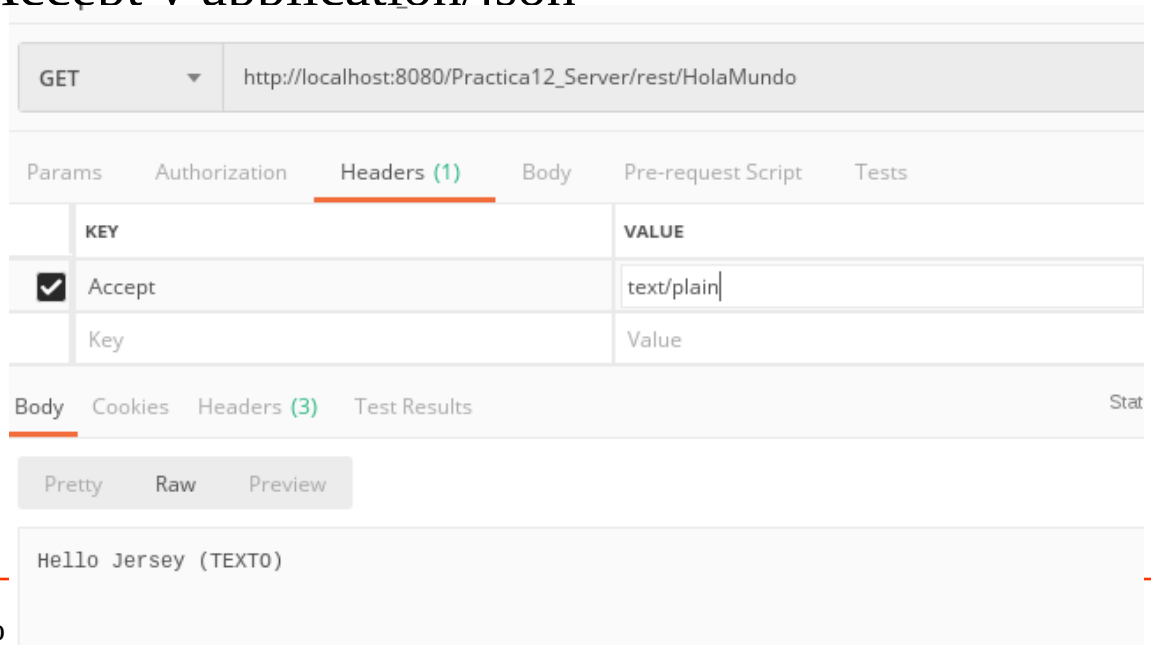
Como probar servicio web REST

- Uso mediante plugins de firefox restClient
  - Seleccionamos get
  - Introducimos url:
    - ✓ http://localhost:8080/Ejemplo_RESTJerseyExample/rest/HolaMundo
  - En headers introducimos:
    - ✓ Accept y text/plain
    - ✓ Accept y application/xml
    - ✓ Accept y application/json



Como probar servicio web REST

- Instalación de Postman (https://www.getpostman.com/downloads/)
  - Seleccionamos get
  - Introducimos url:
    - ✓ http://localhost:8080/Ejemplo_RESTJerseyExample/rest/HolaMundo
  - En headers introducimos:
    - ✓ Accept y text/plain
    - ✓ Accept y application/xml
    - ✓ Accept y application/json



CRUD

- Los servicios REST con HTTP se suelen usar para realizar operaciones CRUD sobre nuestra capa de acceso a datos.
- CRUD responde a “Create Read Update Delete” y se usa para referirse a operaciones de mantenimiento de datos.
- En este estilo de diseño existen dos tipos de recursos: entidades y colecciones.
  - Las colecciones actuan como listas o contenedores de entidades, y en el caso puramente CRUD se suelen corresponder con tablas de base de datos.
  - Las entidades son ocurrencias o instancias concretas, que viven dentro de una colección.

CRUD y operaciones

- El navegador emplea normalmente los métodos HTTP: GET, POST, PUT, DELETE, HEAD y OPTIONS
  - GET:
    - Se usa para obtener información del servidor por tanto no debe cambiar el estado del servidor, es decir, no debe hacer ninguna modificación a cualquier archivo que en éste se encuentre. En términos de CRUD, GET sería el Read (Leer).
  - POST
    - Es el encargado de crear un nuevo recurso y, por consiguiente, modificar el estado del servidor. En términos de CRUD, el método POST se confunde en ocasiones con el método PUT, aunque la diferencia radica en una cualidad que se llama idempotencia.
      - La capacidad que tenga un ente (este caso el método) de realizar una misma operación varias veces, y obtener el mismo resultado que si solo se hiciese una vez.
      - Por este motivo se dice que es el encargado de CREATE del CRUD.

CRUD y operaciones

- Métodos HTTP:
  - PUT
    - Cuando se quiere hacer una operación que haga una acción sobre el estado del servidor, pero que la siguiente vez que suceda, no cambie ese estado. El método PUT con la acción del CRUD, UPDATE (actualizar).
  - DELETE
    - Debe ser usado para borrar un recurso del servidor. En cuanto a CRUD, DELETE sería la acción, que lleva su mismo nombre, DELETE (borrar).
  - HEAD
    - Se usa para obtener la cabecera de respuesta que devuelve el servidor al hacer una petición sobre éste
  - OPTIONS
    - Este método se usa para saber que otros métodos HTTP están disponibles, para determinado recurso, en el servidor.

CRUD - Colecciones

- Las colecciones actuan como listas o contenedores de entidades, y en el caso puramente CRUD se suelen corresponder con tablas de base de datos.
- Normalmente su URI se deriva del nombre de la entidad que contienen. Por ejemplo, http://localhost/rest/productos sería una buena URI para la colección de todos los productos dentro de un sistema.

Método HTTP	Operación
GET	Leer todas las entidades dentro de la colección
PUT	Actualización mútiple y/o masiva
DELETE	Borrar la colección y todas sus entidades
POST	Crear una nueva entidad dentro de la colección

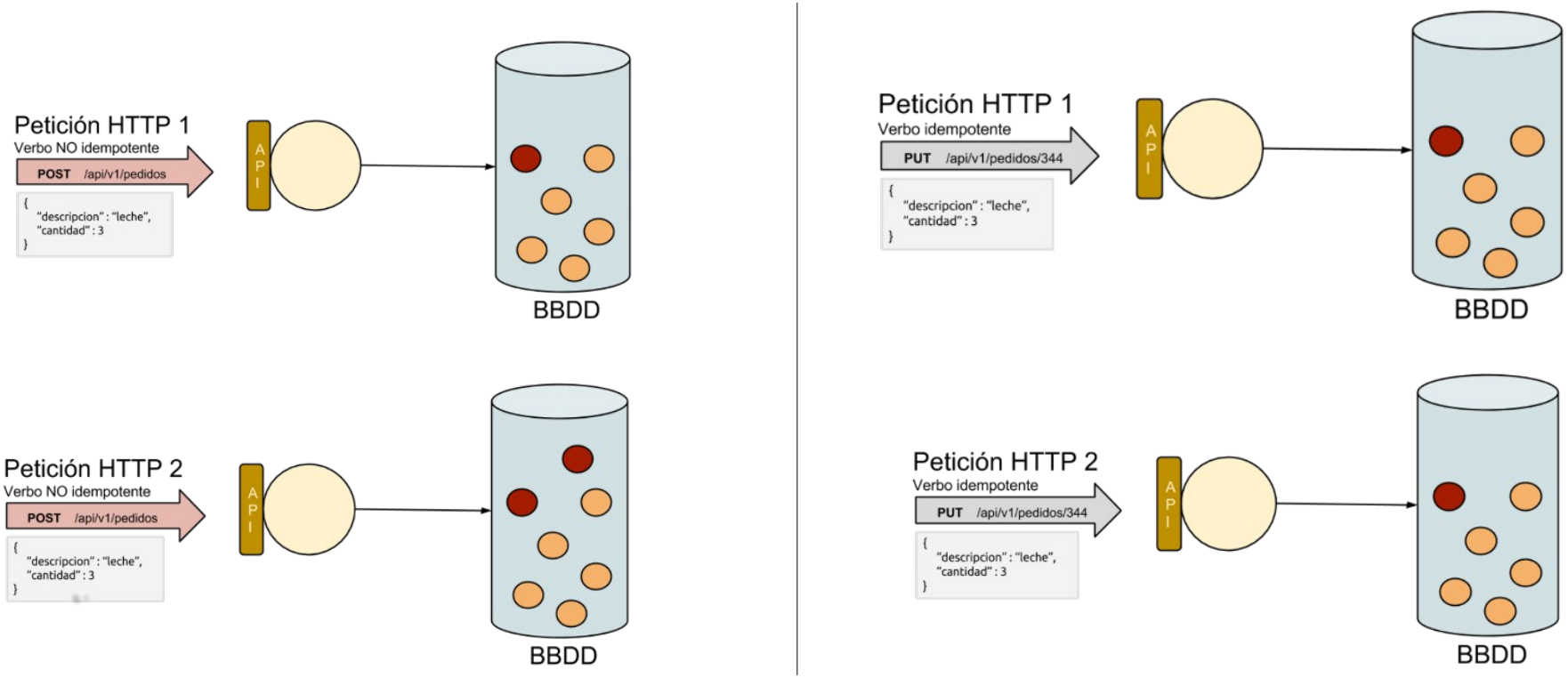
CRUD - Entidades

- Las entidades son ocurrencias o instancias concretas, que viven dentro de una colección.
- La URI de una entidad se suele modelar concatenado a la URI de la colección correspondiente un identificador de entidad. Este identificador necesita ser único dentro de dicha colección.
  - http://localhost/rest/productos/XX445YY sería el libro cuyo identificador es XX45YY.
- Normalmente se suele usar la siguiente convención a la hora de mapear métodos HTTP a operaciones cuando se trabaja con entidades.

Método HTTP	Operación
GET	Leer los datos de una entidad en concreto
PUT	Actualizar una entidad existente o crearla si no existe
DELETE	Borrar una entidad en concreto
POST	Añadir información a una entidad ya existente

Interfaz uniforme y restringido

- REST y el principio de idempotencia
  - Post es idempotente (al enviar la misma petición de creación al mismo recurso con los mismos parámetros) el sistema cambia
  - PUT no es idempotente (al enviar la misma petición de creación al mismo recurso con los mismos parámetros) el sistema no cambia (actualiza)



Obtenido de https://www.adictosaltrabajo.com/2015/06/29/rest-y-el-principio-de-idempotencia/

Orientado a la representación

- Cada vez que un servicio es direccionado a través de una URI, hay un intercambio de representaciones de un recurso entre cliente y servicio
  - Un GET recibe una representación del estado actual de un recurso
  - PUT o POST pasan una representación del recurso al servidor de forma que su estado cambia

- Tipos de representación
  - XML, JSON, YAML, texto plano, HTML,...

Orientado a la representación

- La representación va en el cuerpo del mensaje HTTP
  - La cabecera **Content-type** indica el tipo de formato
  - Cabecera MIME (Multipurpose Intertet Mail Extension)

```
type/subtype;name=value;name=value...
```

```
text/plain
text/html
application/xml
text/html; charset=iso-8859-1
```

- HTTP permite la negociación automática de formato
  - La cabecera **Accept** lista las preferencias de formato en la respuesta
  - Algo más sutil: diferentes versiones del servicio
    - ✓ application/vnd+xml: versión antigua
    - ✓ application/vnd+xml;version=1.1: nueva versión

Comunicación sin estado

- ¿Significa que la aplicación no tiene estado?
  - No! Se refiere a **que no se almacena datos de sesión del cliente** en el servidor
- El servidor sólo almacena y controla los estados de los recursos que proporciona
- Si se necesita datos de sesión, se deben mantener en el cliente y enviarse al servidor en cada petición
- Esto aumenta enormemente la escalabilidad del sistema

¿Por qué es importante un interfaz uniforme?

- Familiaridad
  - Si se tienen URIs que apuntan a un servicio, ya se sabe qué métodos están disponibles en ese recurso
    - ✓ No se necesitan ficheros de descripción de recursos, ni stubs...
    - ✓ Sólo un cliente HTTP
- Interoperabilidad
  - HTTP es muy ubicuo: prácticamente todo lenguaje dispone de librerías para su utilización
  - Si hay un servicio web sobre HTTP, habrá más potenciales “clientes”
- Escalabilidad
  - Dado el conjunto pequeño de operaciones, el comportamiento de los usuarios es muy predecible y eso permite incorporar importantes mecanismos para mejorar el rendimiento
  - Cache en operaciones de GET
  - Control de duplicados en PUT y DELETE

Rest

- Ventajas de rest
  - Mejores tiempos de respuesta y disminución de carga en servidor
  - Mayor escalabilidad al no requerir mantenimiento de estado
  - Facilita desarrollo de clientes
  - Mayor estabilidad frente a futuros cambios
  - Permite evolución independiente de los tipos de documentos
  - La creación de nuevos tipos de documentos no afecta a los anteriores

Comparando ambas soluciones

- Ventajas de REST
  - Ligero: no hace falta mucho XML de configuración.
  - Resultados legibles.
  - Fácil de implementar: no hacen falta herramientas específicas.
  -
- Ventajas de SOAP
  - Fácil de consumir
  - Rigido: tipado fuerte, sigue un contrato
  - Herramientas de desarrollo

SOA
Service Oriented Architecture

Qué es SOA?

- SOA <=> Service Oriented Architecture
- SOA no es un producto, ni un lenguaje, ni una tecnología concreta, ni....
- SOA es una **arquitectura orientada a servicios** que permite la **interoperabilidad entre entornos heterogéneos** y concibe las aplicaciones como colecciones de servicios.
- Es una **estrategia tecnológica** en la cual se utilizan distintos servicios disponibles en una red.
- Estos servicios se comunican entre si para transferirse datos, ejecutar tareas o coordinar una actividad.
  - Evita el re-desarrollo de software por las empresas → Una empresa lo lleva a cabo y lo ofrece como servicio
  - Pequeñas empresas pueden re-ofrecer servicios como si fueran empresas “grandes”
  - ...

Qué es SOA? Antecedentes

- Sockets
  - Permite intercambiar cualquier flujo de datos, generalmente de manera fiable y ordenada, entre dos ordenadores.
- RMI (Java Remote Method Invocation)
  - Mecanismo para invocar un método de manera remota (Java)
  - Forma parte del estándar de ejecución de Java y proporciona un mecanismo simple para la comunicación de servidores en aplicaciones distribuidas basadas exclusivamente en Java.
  - Si se requiere comunicación entre otras tecnologías debe utilizarse CORBA o SOAP
- RPC Remote Procedure Call
  - Protocolo que permite a un programa ejecutar código en otra máquina remota
  - El control de la comunicación es realizado por el protócolo.
  - Gran avance sobre los sockets → el programador no tenía que estar pendiente de las comunicaciones.

Qué es SOA?

- Implementar SOA
  - SOA no es “Web Services”.
  - SOA es un concepto que ha sido usado durante años.
    - Sockets
    - Mensajería
    - RMI
    - RPC
    - Corba
    - ….
  - La principal característica de SOA es que es una arquitectura con acoplamiento débil, esto significa, que el cliente de un servicio es esencialmente independiente de la construcción de este servicio
  - Una implementación de SOA son los “Web Services”
    - Web Services es un servicio que comunica con los clientes a través de un conjunto estándar de protocolos y tecnologías

Qué es SOA?

- Podemos distinguir tres aproximaciones:
  - XML-RPC(Extensible Markup Language–Remote Procedure Call)
  - Servicios web basados en SOAP
    - ✓ Estilo RPC: Permitir al programador invocar operaciones remotas de forma muy parecida a como si fuesen locales.
    - ✓ Se basa en nuevas tecnologías específicas: SOAP, WSDL,UDDI
  - REST (Representational Style Transfer)
    - ✓ Es un nuevo estilo arquitectónico, no una tecnología.
    - ✓ Se basa fundamentalmente en tecnologías existentes: HTTP 1.1 y XML y/o JSON

XML-RPC

- **XML-RPC** es un protocolo de llamada a procedimiento remoto que usa XML para codificar los datos y HTTP como protocolo de transmisión de mensajes.
  - Es un protocolo muy simple unos cuantos tipos de datos y comandos útiles, además de una descripción completa de corta extensión.
  - La simplicidad del XML-RPC contrasta con la mayoría de protocolos RPC que tiene una documentación extensa y requiere considerable soporte de software para su uso.
  - Fue creado por Dave Winer de la empresa UserLand Software en asociación con Microsoft en el año 1998.
  - Se convirtió en lo que es actualmente conocido como SOAP.
  - Una diferencia fundamental es que en los procedimientos en SOAP los parámetros tienen nombre y no interesa su orden, no siendo así en XML-RPC.

Invocación XML-RPC [1]

```
<?xml version="1.0"?>
  <methodCall>
    <methodName>org.wikipedia.intercambioDatos</methodName>
    <params>
      <param>
         <value><i4>360</i4></value>
      </param>
      <param>
         <value><i4>221</i4></value>
      </param>
    </params>
  </methodCall>
```

Respuesta a la invocación:

```
<?xml version="1.0"?>
 <methodResponse>
   <params>
     <param>
       <value><string>Intercambio datos nro. 360 por 221</string></value>
     </param>
   </params>
 </methodResponse>
```


1 Wikipedia

SOAP
Simple Object Access Protocol

Introducción a SOAP

- SOAP (siglas de Simple Object Access Protocol) es un protocolo estándar que establece como pueden comunicarse dos equipos por medio de intercambio de datos XML.
- Su precedesor fue creado por Dave Winer en 1998, y fue llamado XML-RPC.
- SOAP fue creado por Microsoft, IBM y otros. Está actualmente bajo la W3C.
- Es uno de los protocolos utilizados en los servicios Web.

Arquitectura de SOAP

- Elementos en Servicios Web
  - Servidor, aloja y ejecuta el Servicio Web
  - Registro, asocia servicios y servidores
  - Cliente, pregunta al registro y llama al servicio



Arquitectura de SOAP

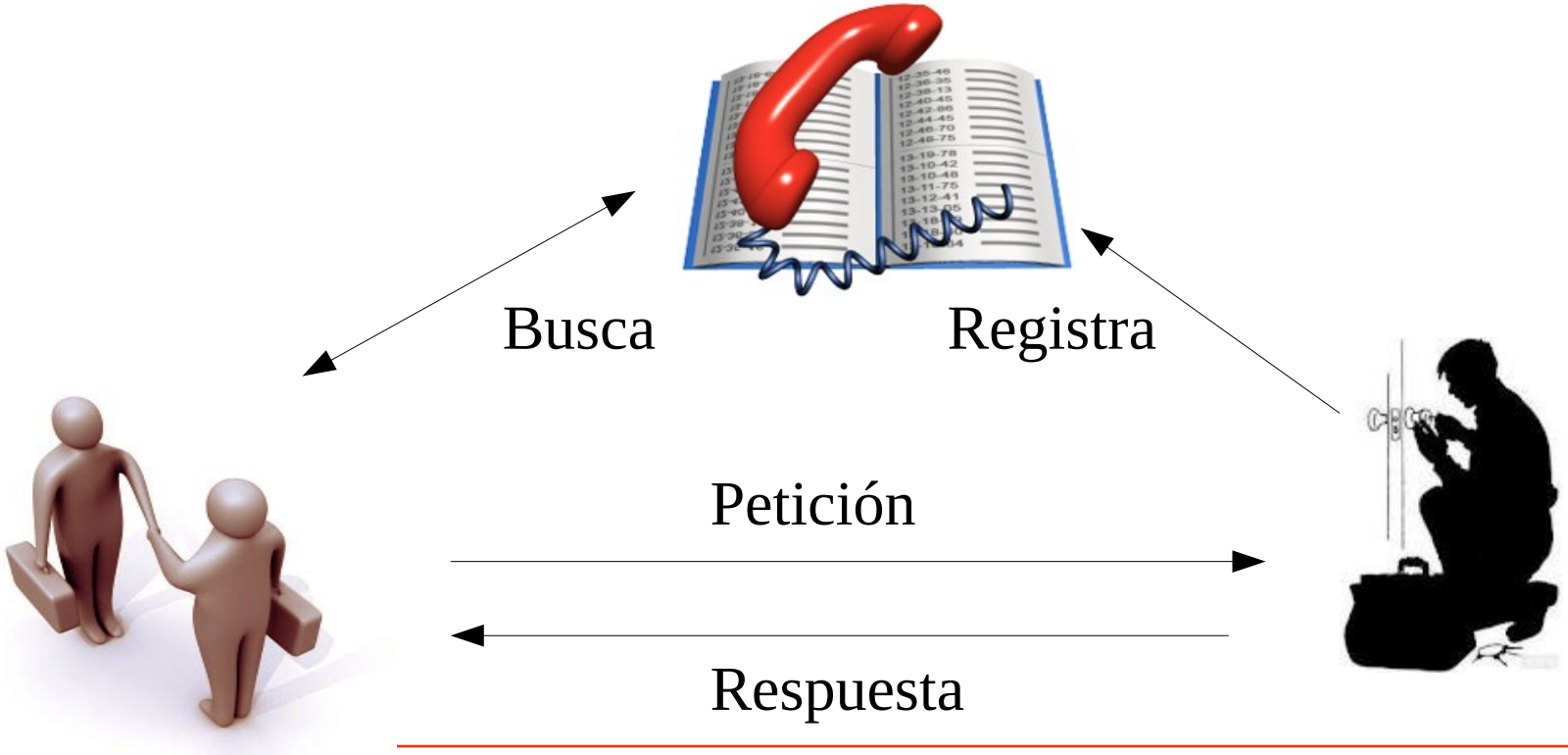
- Solicitud de servicio
  - Se debe identificar en el servidor los servicios de los que dispone
  - Una vez identificado el servicio que quiere, se tiene que obtener una descripción del servicio.
    - Si éste es un servicio de SOAP, es probable que encuentre un documento de WSDL.
    - Si es un servicio XML-RPC, debe haber algunas instrucciones humano-leíbles para la integración.
  - Aplicación del cliente.
    - Crear un XML-RPC
    - Cliente SOAP en el lenguaje de su opción.
  - Finalmente, ejecute su aplicación del cliente para invocar el servicio Web actualmente.

Arquitectura de SOAP

- Proveedor de servicio
  - Desarrollar la funcionalidad que quiere ofrecer
  - Desarrollar una envoltura de servicio a su funcionalidad del núcleo (XML-RPC o una envoltura de servicio SOAP).
  - Proporcionar una descripción de servicio.
    - ✓ SOAP, crear un archivo de WSDL.
    - ✓ XML-RPC la creación de instrucciones humano-leíbles.
  - Desplegue del servicio → Instalación en un servidor de aplicaciones
  - Publicar el servicio → a un directorio de UDDI .

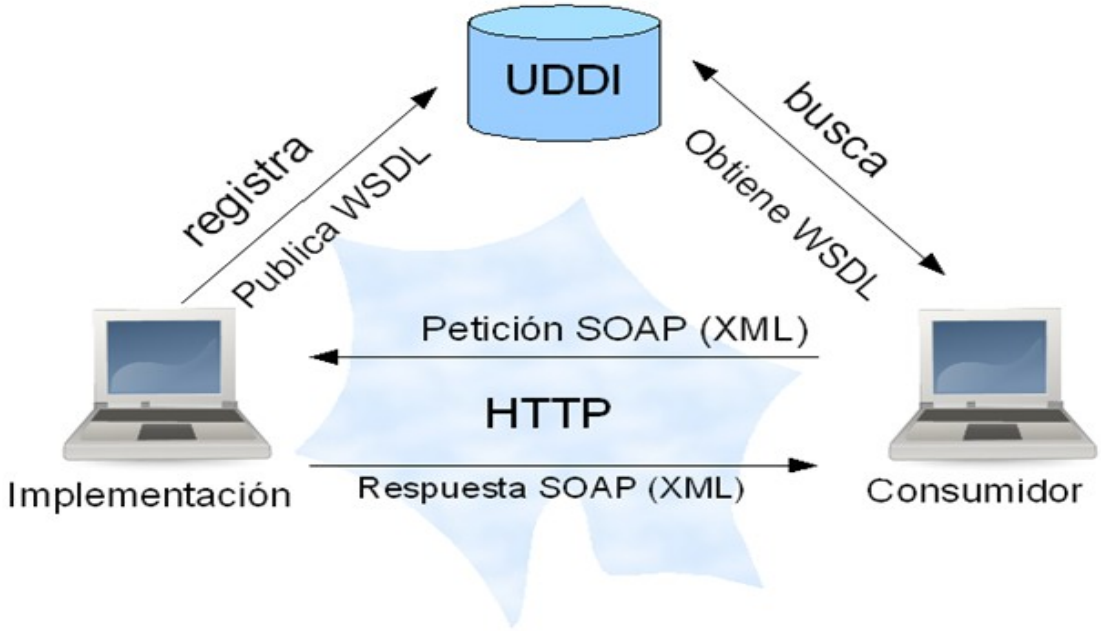
Arquitectura de un servicio web

- Esta filosofía existe en otros negocios clásicos
  - Proveedores de Servicios (Cerrajeros)
  - Consumidores de Servicios (Clientes)
  - Localización de Servicios (Páginas amarillas/Páginas Blancas, Internet, ….)



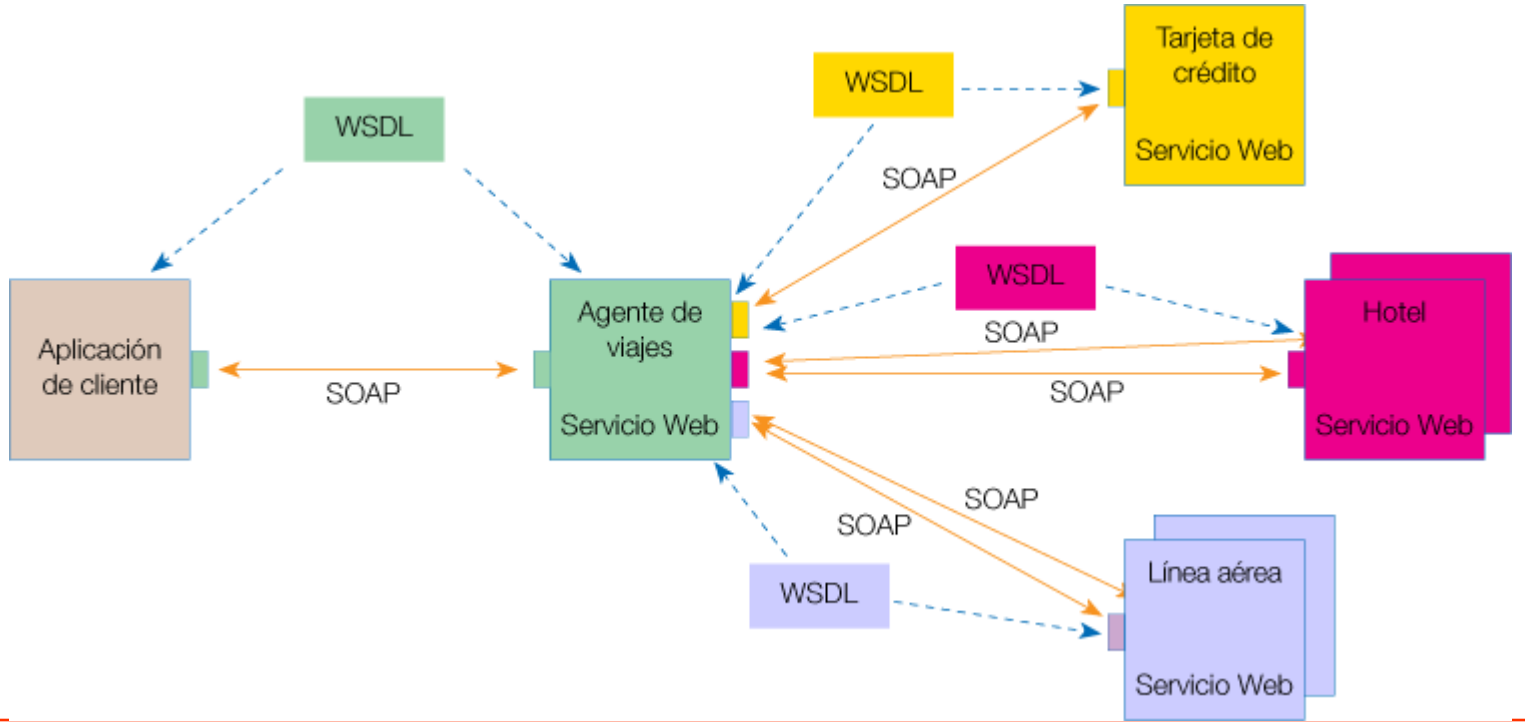
Arquitectura de un servicio web

- Elementos en Servicios Web: Tecnología subadyacente
  - Publicados y localizados vía UDDI (Universal Description, Discovery and Integration).
  - Descritos usando WSDL (WebService Description Language).
  - Invocados vía SOAP (Simple Object Access Protocol) sobre HTTP.



Arquitectura de un servicio web

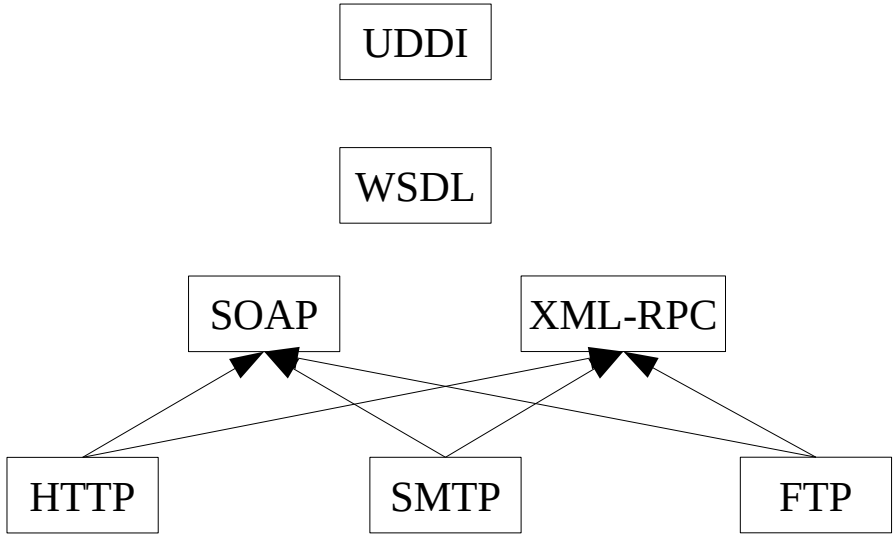
- Elementos en Servicios Web: Tecnología subadyacente
  - Publicados y localizados vía UDDI (Universal Description, Discovery and Integration).
  - Descritos usando WSDL (WebService Description Language).
  - Invocados vía SOAP (Simple Object Access Protocol) sobre HTTP.



Arquitectura de un servicio web

- Capas que intervienen:

Descubrimiento	UDDI
Descripción	WSDL
Mensajería XML	XML-RPC, SOAP, XML
Transporte	HTTP, SMTP, FTP



Componentes de SOAP

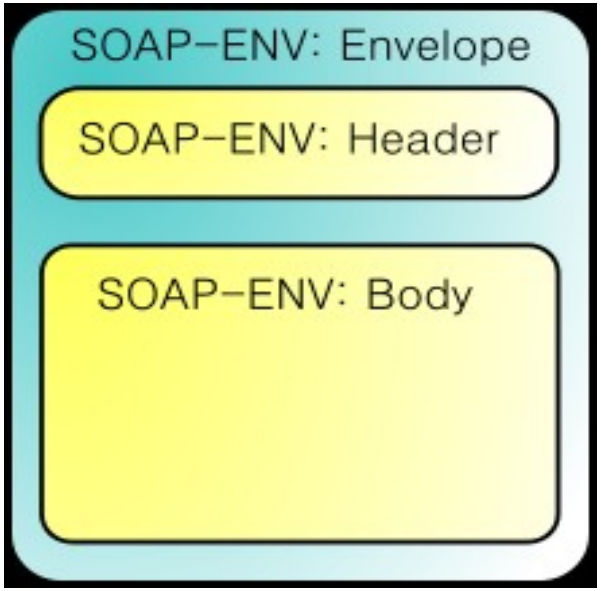
- XML (eXtensible Markup Language)
  - Usado tanto para para el intercambio de datos como para describir las funcionalidades
- SOAP (Simple Object Access Protocol)
  - Plataforma para el intercambio de servicios sobre la red.
  - Protocolo con el que hablan los servicios.
  - Permite a las aplicaciones invocar métodos de objetos remotos
  - Actúa sobre multitud de protocolos (HTTP, SMTP, …)
- UDDI (Universal Description, Discovery and Integration)
  - Permite descubrir con quién comunicarse y dónde
  - Desacopla la interacción entre consumidor/proveedor
- WSDL (Web Service Description Language)
  - Describe los mensajes SOAP que definen un servicio Web en particular
  - Utiliza un IDL (Interface Definition Language) de servicios

Introducción a SOAP

- SOAP (Simple Object Access Protocol)
  - La especificación de SOAP define una estructura de la mensajería para intercambiar los datos de XML formateados en Internet.
  - Es simple, fácil de desarrollar y completamente neutral con respecto al sistema operativo, lenguaje de programación o la plataforma.
  - SOAP esta diseñado para proporcionar un protocolo de comunicaciones independiente, abstracto capaz de conectar dos o más sitios remotos.

SOAP

- SOAP esta formado principalmente por tres partes
  - Envelope: Define el inicio y el fin del mensaje (envelope=sobre).
  - Header:
    - Provee de un mecanismo para extender un mensaje SOAP
    - Elemento opcional que contiene meta-información .
    - Se puede dividir en bloques.
  - Body:
    - Elemento obligatorio con los datos del mensaje.
    - Se puede dividir en bloques.



SOAP

- SOAP esta formado principalmente por tres partes

```
<?xml version="1.0"?>
<soap:Envelope
   xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
   soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">

      <soap:Header>
         ...
      </soap:Header>

      <soap:Body>
         ...
         <soap:Fault>
            ...
         </soap:Fault>

      </soap:Body>
</soap:Envelope>
```

Es un documento XML

Comienza el mensaje

Espacio de Nombre

donde se encuentran definidos los tipos de datos

Cabecera-opcional Metainformación

Error

Cuerpo del Mensaje

SOAP. Ejemplo



Petición

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
     <getProductDetails xmlns="http://warehouse.example.com/ws">
       <productId>827635</productId>
     </getProductDetails>
   </soap:Body>
</soap:Envelope>
```

Respuesta

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
   <soap:Body>
     <getProductDetailsResponse xmlns="http://warehouse.example.com/ws">
       <getProductDetailsResult>
         <productName>Toptimate 3-Piece Set</productName>
         <productId>827635</productId>
         <description>3-Piece luggage set.  </description>
         <price>96.50</price>
         <inStock>true</inStock>
       </getProductDetailsResult>
     </getProductDetailsResponse>
   </soap:Body>
</soap:Envelope>
```

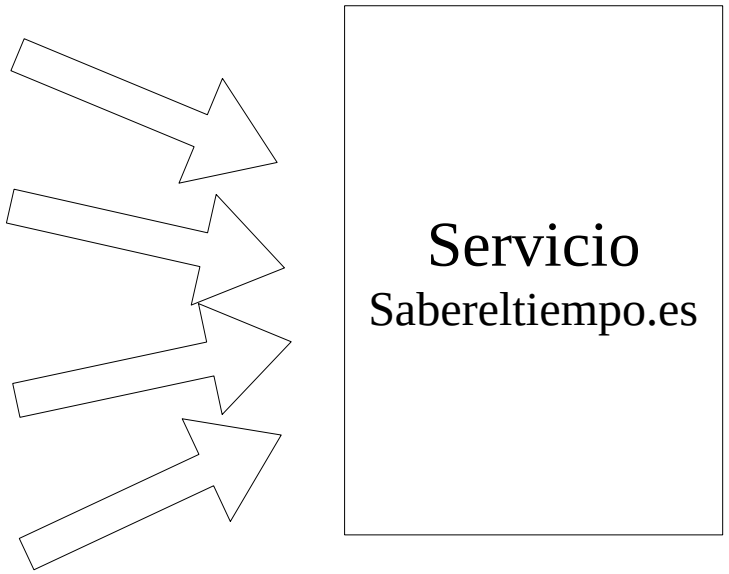


Introducción a WSDL

- WDSL: Descripción de los servicios (especialmente sus interfaces) de forma estándar
  - Formato XML que se utiliza para describir servicios Web .
  - La interface especifica cada método, con sus parámetros de entrada y salida.
    - Si el estilo de interacción es RPC, estos mensajes llevan los parámetros de entrada y salida del procedimiento invocado.
    - Si el estilo de interacción es documento, estos mensajes llevan los datos en si que deben ser validados con el esquema.
    - Un fichero WSDL se puede compilar para el lenguaje de programación apropiado y generar los stubs y capas intermedias necesarias para hacer las llamadas Web de forma transparente.

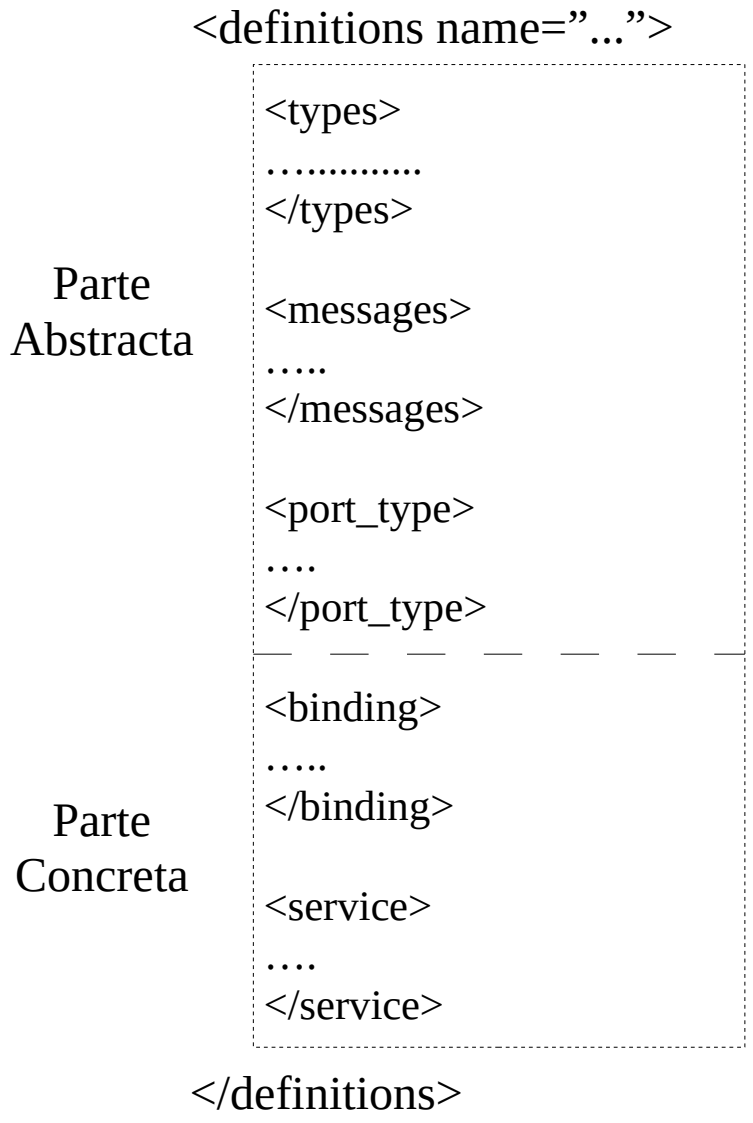
¿Qué es WSDL?

- WSDL ↔ Web Services Description Language
  - Describe cuatro piezas principales:
    - ✓ Información sobre la interfaz visible y utilizable.
    - ✓ Tipos de datos de los mensajes intercambiados (peticiones, respuestas, errores).
    - ✓ Información de cómo transportar los mensajes en la comunicación.
    - ✓ Direcciones para localizar los servicios.
  - WSDL representa una especie de contrato entre el proveedor y el que solicita.



Introducción a WSDL

- Documento WSDL en pocas palabras
  - <definitions>, contiene la definición de uno o más servicios.
  - <message> y <portType>, qué operaciones provee el servicio
  - <binding>, cómo se invocan las operaciones
  - <service>, dónde se ubica el servicio
  - <documentation>, puede contener información del servicio para el usuario



Introducción a WSDL

```
<?xml version="1.0"?>
<definitions name="Procurement"
  targetNamespace="http://example.com/procurement/definitions"
  xmlns:tns="http://example.com/procurement/definitions"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns="http://schemas.xmlsoap.org/wsdl/" >

  <message name="OrderMsg">
    <part name="productName" type="xs:string"/>
    <part name="quantity" type="xs:integer"/>
  </message>

  <portType name="procurementPortType">
    <operation  name="orderGoods">
      <input message = "OrderMsg"/>
    </operation>
  </portType>

  <binding name="ProcurementSoapBinding" type="tns:procurementPortType">
    <soap:binding style="document"
        transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="orderGoods">
      <soap:operation soapAction="http://example.com/orderGoods"/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>

  <service name="ProcurementService">
    <port name="ProcurementPort" binding="tns:ProcurementSoapBinding">
      <soap:address location="http://example.com/procurement"/>
    </port>
  </service>
</definitions>
```

abstract part

messages

operation and port type

concrete part

binding

port and service

Copyright Springer Verlag Berlin Heidelberg 2004

Introducción a UDDI

- UDDI (Universal Description, Discovery and Integration).
  - Catálogo de negocios de Internet
  - El registro en el catálogo se hace en XML.
  - UDDI es una iniciativa industrial abierta (sufragada por la OASIS) en el contexto de los servicios Web.
  - El registro de un negocio en UDDI tiene tres partes:
    - Páginas blancas - dirección, contacto y otros identificadores conocidos.
    - Páginas amarillas - categorización industrial basada en taxonomías.
    - Páginas verdes - información técnica sobre los servicios que aportan las propias empresas.
  - Características de UDDI. Dos categorías de API:
    - De publicación
    - D consulta

Plataformas de desarrollo de SOAP

- Cada plataforma tiene diferentes formas de implementar los servicios
  - .Net
  - Java
    - ✓ Axis (sucesor del Apache SOAP 2.2, xml.apache.org/axis)
    - ✓ CXF
    - ✓ SUN: Java Web Services Developer Pack (JWSDP)
    - ✓ CapeConnect (www.capeclear.com)
    - ✓ Glue Standard (www-themindelectric.com)
    - ✓ IONA Orbix E3A XMLBus (www.iona.com)
    - ✓ WASP Server for Java (www.systinet.com)